

Tema 4: Realización de consultas

Creación de vistas

Contenido

1.	<u>Introducción.....</u>	<u>3</u>
2.	<u>Creación de vistas.....</u>	<u>3</u>
3.	<u>Consulta de vistas existentes.....</u>	<u>6</u>
4.	<u>Consulta estructura de una vista.....</u>	<u>7</u>
5.	<u>Borrado de vistas.....</u>	<u>8</u>
6.	<u>Ventajas.....</u>	<u>8</u>

1. Introducción

En este tema se va a cubrir la creación de vistas. Una vista es una consulta de datos que se comporta como una tabla virtual, la diferencia sobre las tablas derivadas es que la vista existe fuera de la consulta, es decir, la tabla virtual existe en la base de datos y se comporta como otra tabla más.

2. Creación de vistas

La sintaxis es:

```
create view nombre_vista as  
select ...  
from  
...
```

Por ejemplo, con las siguientes tablas:

```
create table clientes  
    ( id_cliente int primary  
      key, nombre varchar(50)  
    );
```

```
create table pedidos  
    ( id_pedido int primary  
      key, id_cliente int,  
      total decimal(10,2),
```

IES Isidra de Guzmán	
----------------------	--

```
foreign key (id_cliente) references clientes(id_cliente)
);
```

```
insert into clientes values
```

```
(1, 'Ana'),
(2, 'Luis'),
(3, 'María');
```

```
insert into pedidos values
```

```
(101, 1, 500.00),
(102, 1, 500.00),
(103, 2, 100.00),
(104, 3, 50.00),
(105, 3, 50.00),
(106, 3, 50.00),
(107, 3, 50.00),
(108, 3, 50.00),
(109, 3, 50.00),
(110, 3, 50.00),
(111, 3, 50.00);
```

```
create view clientes_pedidos as
```

```
select cl.nombre as nombre,
pe.id_pedido as id_pedido,
pe.total as total
from
clientes cl
inner join pedidos pe
on cl.id_cliente=pe.id_cliente;
```

Se puede realizar una consulta como si fuera una tabla, por ejemplo, pedidos de más de 300 euros:

```
select *  
from  
clientes_pedidos  
where total>300;
```

+	-----	+	-----	+	-----	+
	nombre		id_pedido		total	
+	-----	+	-----	+	-----	+
	Ana		101		500.00	
	Ana		102		500.00	
+	-----	+	-----	+	-----	+

Al ser una tabla dinámica si ahora se insertara un nuevo pedido la vista se refresca automáticamente, por ejemplo:

```
insert into pedidos  
(id_pedido, id_cliente, total)  
values  
(200, 3, 600);
```

```
select *  
from  
clientes_pedidos  
where total>300;
```

IES Isidra de Guzmán			
+	+	+	+
nombre	id_pedido	total	
+	+	+	+
Ana	101	500.00	
Ana	102	500.00	
María	200	600.00	
+	+	+	+

Se pueden crear vistas sobre vistas, aunque se corre el riesgo de que sea difícil trazar el origen de los datos y corregir posibles anomalías. Por ejemplo, si se desea una vista que tenga el cliente con el pedido mayor:

```
create view cliente_maximo as
select
nombre,
total
from clientes_pedidos
where total =
(select
max(total)
from
clientes_pedidos);
```

3. Consulta de vistas existentes

En MySQL mediante la siguiente consulta se pueden obtener las vistas existentes:

```
show full tables
```

IES Isidra de Guzmán	
----------------------	--

```
where table_type='VIEW';
```

Tables_in_ut4_vistas	Table_type	
cliente_maximo	VIEW	
cliente_pedidos	VIEW	
clientes_pedidos	VIEW	

4. Consulta estructura de una vista

Se pueden utilizar los mismos comandos que en una tabla: describe y show create view ...

```
desc clientes_pedidos;
```

Field	Type	Null	Key	Default	Extra	
nombre	varchar(50)	YES		NULL		
id_pedido	int	NO		NULL		
total	decimal(10,2)	YES		NULL		

```
show create view clientes_pedidos
```


```

+-----+
| IES Isidra de Guzmán |
+-----+
+-----+
+-----+
| View | Create View
| character_set_client | collation_connection |
+-----+
+-----+
+-----+
+-----+
+-----+
+-----+
+-----+
+-----+
| clientes_pedidos | CREATE ALGORITHM=UNDEFINED DEFINER=`alumno`@`localhost`
SQL SECURITY DEFINER VIEW `clientes_pedidos` AS select `cl`.`nombre` AS
`nombre`,`pe`.`id_pedido` AS `id_pedido`,`pe`.`total` AS `total` from
(`clientes` `cl` join `pedidos` `pe` on((`cl`.`id_cliente` =
`pe`.`id_cliente`))) | utf8mb4 | utf8mb4_0900_ai_ci |
+-----+
+-----+
+-----+
+-----+
+-----+
+-----+
+-----+
+-----+

```

5. Borrado de vistas

La sintaxis es:

```
drop view nombre_vista;
```

```
drop view cliente_maximo;
```

6. Ventajas

IES Isidra de Guzmán	
----------------------	--

Las ventajas que se pueden destacar son:

- Son más rápidas que las consultas, al estar ya precompiladas/interpretadas.
 - Mejoran la comprensión sobre las tablas derivadas ya que separan claramente una consulta de otra.
 - Proveen de más seguridad: se pueden otorgar permisos de `select` sobre una vista en lugar de sobre una tabla, se puede restringir el acceso a columnas, etc.

Actividad 1

Dada la siguiente tabla y datos:

```
create table productos
( id_producto int primary
  key, nombre varchar(50),
  categoria varchar(50)
);
```

```
create table ventas
( id_venta int primary
  key, id_producto int,
  cantidad int,
  foreign key (id_producto) references productos(id_producto)
);
```

```
insert into productos (id_producto, nombre, categoria) values
(1, 'laptop', 'tecnologia'),
(2, 'mouse', 'tecnologia'),
(3, 'silla', 'hogar'),
```

IES Isidra de Guzmán	
----------------------	--

```
(4, 'mesa', 'hogar'),
(5, 'balon', 'deportes');
```

```
insert into ventas (id_venta, id_producto, cantidad) values
(1, 1, 5),
(2, 1, 3),
(3, 2, 10),
(4, 3, 7),
(5, 3, 2),
(6, 4, 4),
(7, 5, 12);
```

Realizar las siguientes vistas:

- Inner join de las 2 tablas.
- CREATE OR REPLACE VIEW vista_productos_ventas AS SELECT p.nombre, p.categoria, v.cantidad FROM productos p INNER JOIN ventas v ON p.id_producto = v.id_producto;
- Total ventas por producto y categoría.
- CREATE OR REPLACE VIEW vista_totales_producto AS SELECT nombre, categoria, SUM(cantidad) AS total_vendido FROM vista_productos_ventas GROUP BY nombre, categoria;
- Máximo ventas por categoría.
- CREATE OR REPLACE VIEW vista_max_categoria AS SELECT categoria, MAX(total_vendido) AS max_venta FROM vista_totales_producto GROUP BY categoria;
 - Máximo ventas de cada producto en su categoría (se puede hacer inner join de las 2 vistas anteriores)
 - CREATE OR REPLACE VIEW vista_final_ganadores AS SELECT t.nombre, t.categoria, t.total_vendido FROM vista_totales_producto t INNER JOIN vista_max_categoria m ON t.categoria = m.categoria AND t.total_vendido = m.max_venta;
- 4. Comprobación final:
- SELECT * FROM vista_final_ganadores;